

Notes on The C Programming Language

Nicoll Douglas

April 8, 2026

Contents

1	Types, Operators & Expressions	1
1.1	Variable Names	1
1.2	Data Types and Sizes	2
1.3	Constants	2
1.3.1	Integer Constants	2
1.3.2	Character Constants	3
1.3.3	Constant Expressions	4
1.3.4	String Constants	4
1.3.5	Enum Constants	4
1.4	Declarations	5
1.5	Operators	6
1.6	Type Conversions	7
1.6.1	Integer Promotion	7
1.6.2	Order of Type Conversion Operations	8
1.6.3	Sign Extension and Zero Extension	8
1.6.4	Truncation	8
1.6.5	Floats	8
1.6.6	Casting	8

1 Types, Operators & Expressions

1.1 Variable Names

- Must only contain letters, digits, and underscores.
- Must start with a letter or underscore¹.

¹Library functions often start with underscores so avoid.

1.2 Data Types and Sizes

Type	Size (bytes)	Description
char	1	Holds one character
short / short int	2 ²	Smaller integer
int	4 ³	Natural integer size on host machine
long / long int	8 ⁴	Bigger integer
float	4	Single-precision floating point
double	8	Double-precision floating point
long double	16 ⁵	Extended-precision floating-point

Table 1: Type, size, and description of primitive data types in C

- The **signed** or **unsigned** qualifiers may be applied to **char** or any integer type.
- **unsigned** integers are positive or 0.
- **unsigned** integers obey the laws of arithmetic module 2^n where n is the number of bits in the data type.
- **int**, **short**, and **long** are all signed by default.
- **char** can be **unsigned** or **signed** by default (not mandated in C standard) but is usually **signed**.

1.3 Constants

1.3.1 Integer Constants

Suffixes Integer constants may contain the given suffixes indicated below.

²C standard minimum 2 bytes, typically 2 bytes.

³C standard minimum 2 bytes and at least as big as **short**; typically 4 bytes.

⁴C standard minimum 4 bytes and at least as big as **int**; typically 8 bytes.

⁵C standard at least as long as **double**; usually 8, 12 or 16 bytes.

Suffix	Type	Examples
None, only digits	int (long if too big)	1234
l or L	long	1234L
u or U	unsigned int (unsigned long if too big)	1234U
ul or UL	unsigned long	1234UL
None, with decimal or scientific	double	1.234, 1e-5
f or F	float	1.234F
l or L, with decimal or scientific	long double	1.234L, 1e-5L

Table 2: Data type and examples of integers using the given suffix

Prefixes Integer constants may contain the given prefixes below. These can also be used in conjunction with appropriate suffixes (those available for integers).

Prefix	Interpretation
None	Decimal
0	Octal
0x or 0X	Hexadecimal

Table 3: Interpretations of integer constants with the given prefix

1.3.2 Character Constants

- Character constants are integers written as one character with single quotes and are equal to the numeric value in the machine's character set.
- Character constants can also participate in arithmetic operations like other integer types.

Escape Sequences Certain characters can be represented by the following escape sequences.

Sequence	Meaning	ASCII Code
\0	Null character	0
\a	Alert (bell) character	7
\b	Backspace	8
\t	Horizontal tab	9
\n	Line feed (newline)	10
\v	Vertical tab	11
\f	Form feed	12
\r	Carriage return	13
\"	Double quote	34
\'	Single quote	39
\?	Question mark	63
\\	Backslash	92
\ooo	Octal code (o = octal digit)	
\xhh	Hex code (h = hex digit)	

Table 4: Different escape sequences, their meaning, and ASCII codes in C

1.3.3 Constant Expressions

- Constant expressions are expressions that involve only constants.
- Such expressions are typically evaluated at compile-time and can be used in any place that a constant can occur.

Example:

```

1 #define LEAP 1;
2
3 int days[31+28+LEAP+31+30+31+30+31+31+30+31+30+31];

```

1.3.4 String Constants

- A string constant (a.k.a. a string literal) is a sequence of 0 or more characters surrounded by double quotes.
- The escape sequences used in characters also apply in strings.
- String constants can be concatenated at compile-time, e.g "hello, world" is equivalent to "hello , " " world".
- This is useful for breaking up long strings across several lines of code.

1.3.5 Enum Constants

- An enum is a list of constant integer values.
- Names in enums *must* be distinct.

- Values in enums don't need to be distinct.
- If values are left unassigned, they increase in order from the leftmost assigned value (the first will be 0 if unassigned).

Examples:

```
1 enum boolean { NO, YES }; // NO=0, YES=1
2 enum letters { A = 10, B, C }; // A=10, B=11, C=12
3 enum letters2 { A, B = 50, C }; // A=0, B=50, C=51
```

1.4 Declarations

- All variable declarations must be at the top of a code block before executable statements and declared before use.⁶
- A declaration contains a list of one or more variables of that type which may optionally be assigned.
- Automatic variables are local variables that are automatically allocated when the program enters the code block they are defined in.
- If a variable is not automatic, its initialization is done only once (before the program starts executing).
- Otherwise, it is initialised each time its code block is entered.
- External and **static** variables are initialised to 0 by default.
- Automatic variables with no initializer are undefined (garbage bits leftover in memory).
- The **const** qualifier can be applied to a variable declaration to specify its value will not be changed (for arrays the element will not be altered).
- **const** can also be used with function arguments to indicate that the function will not change, or reassign the local copy of the value passed.

Automatic variable examples:

```
1 int a, b, c = 10; // a and b are undefined
2 const int d = 20; // will not be changed
```

⁶Specific to C89/90.

1.5 Operators

Precedence Operator precedence is defined according to the following table:

Rank	Category	Operators	Association
1	Postfix	++, -, (), [], ->, .	LTR
2	Prefix	+, -, ++, --, !,	RTL
3	Multiplicative	*, /, %	LTR
4	Additive	+, -	LTR
5	Bit Shift	«, »	LTR
6	Relational	<, <=, >, >=	LTR
7	Equality	==, !=	LTR
8	Bitwise AND	&	LTR
9	Bitwise XOR	^	LTR
10	Bitwise OR		LTR
11	Logical AND	&&	LTR
12	Logical OR		LTR
13	Ternary	?:	RTL
14	Assignment	=, +=, -=, *=, /=, %=, &=, ^=, =, «=, »=	RTL
15	Comma	,	LTR

Arithmetic Operators

- Arithmetic operators consist of the additive and multiplicative operators.
- The % operator cannot be applied to floats or doubles.
- The direction for truncation for / is towards zero.⁷
- The C standard requires that $(a/b) \cdot b + (a \% b) == a$ where / is floor division.
- The sign of the result for % usually takes on the sign of the dividend (a for $a \% b$) as a result of / truncating towards zero.⁸

Relational and Logical Operators

- The numerical value of expressions involving relational and logical operators is 1 if the expression is true or 0 if false.

⁷In the C89/90 standard it is implementation dependent, can be towards zero or negative infinity but typically zero in modern standards.

⁸For truncation towards negative infinity the sign of modulo results take on the sign of the divisor.

Increment and Decrement Operators

- Prefix increment/decrement *modifies the value then consumes*.
- Postfix increment/decrement *consumes the value then modifies*.
- Increment/decrement operators can only be applied to variables (not expressions).

Bitwise Operators

- Bitwise operators may only be used on integer type operands.
- Bitwise AND is often used to mask off some bits (e.g `n &= 0xF` sets all bits to 0 except the 4 lowest order bits).
- Bitwise OR is often used to turn bits on (e.g `n |= 0xF` turns on the 4 lowest order bits).
- Shift operators `<<` and `>>` shift the bits up and down respectively of the left-hand operator by the non-negative amount in the right-hand operator (e.g `2 << 2 = 8` since 0010 becomes 1000).
- Vacant bits after shifting are filled with 0s usually.

Ternary Operator

- The first expression of a ternary operator expression is evaluated.
- If the first is non-zero (true), the second is evaluated and that is the value of the ternary expression, otherwise (if it is false) the third expression is evaluated and *that* is the value of the ternary expression.
- If the second and third expressions differ in type, then type conversion occurs based on the type conversion rules in the next section.

1.6 Type Conversions

- Type conversions take place in things like expressions and assignments.
- Function prototypes enable automatic casting of arguments.

1.6.1 Integer Promotion

- When a smaller rank is used in a binary arithmetic operation, it is automatically promoted to the larger rank used in the expression (promotion).
- So the operation can be performed as if it was on two operands of the same type.
- In an expression, type conversion happens incrementally along the way as the expression is evaluated according to associativity of each operator.
- `chars` and `shorts` are always promoted to `int` (signed) in an expression.

1.6.2 Order of Type Conversion Operations

1. `chars` and `shorts` are promoted to `int` and any other integer promotion goes through.
2. Signs are then compared using the size of the original types before promotion (`int` if originally `char` or `short`).
3. If a is **unsigned** and has size $\geq$ than b , a **signed** type, b becomes unsigned.
4. If a is **unsigned** and has size $<$ than b , a **signed** type, a becomes signed.⁹

1.6.3 Sign Extension and Zero Extension

- When a smaller type is unsigned and promoted, the program performs *zero extension* i.e, fills the new high-order bits with 0s.
- When a smaller type is signed and promoted, the program performs *sign extension* i.e, fills the new high-order bits with whatever the old two's complement sign bit was (0 if positive, 1 if negative).
- Each of these operations preserves the original value but the actual value depends on the new interpretation (i.e signed or unsigned).

1.6.4 Truncation

- Larger integer types are converted to smaller ones by chopping off the excess higher order bits.

1.6.5 Floats

- If integer types are mixed with float types, the float type always wins.
- If the integer type is larger than the float type, precision loss may occur.

1.6.6 Casting

- An expression can be forcibly cast to another type.
- A cast behaves the same way as if an expression was being assigned to a variable of the cast type.
- Casting uses the following syntax:

(type name) expression

⁹The positive signed range of type b usually matches the unsigned range of a , with the type of b having twice the number of bits.